

On-Demand Post Generation

How a single post is created, reviewed, approved, rejected & emailed — explained in plain language

Scope: The client page `/{{brand}}/on-demand/new` and the admin review screen `/admin/qa?tab=on_demand`

Covers: the generation pipeline · every database table & field · hold / approve / reject · regeneration · chain selection · email notifications

Audience: non-technical — written so anyone on the team can follow it

Prepared: 13 June 2026 · based on a direct reading of the live source code (file references included throughout)

What's inside

1. 1 — The big picture (in one minute)
2. 2 — The client page: </{brand}/on-demand/new>
3. 3 — What happens after “Generate” (the AI pipeline)
4. 4 — How chain selection works
5. 5 — The three outcomes: Delivered, Held, Failed
6. 6 — The database: every table & field, and why
7. 7 — The admin review screen: /admin/qa?tab=on_demand
8. 8 — Regeneration (client side & admin side)
9. 9 — Approve, reject & hold — what each button really does
10. 10 — How email notifications work
11. 11 — Reference map (where everything lives in the code)

1 — The big picture (in one minute)

A brand owner wants one custom post — right now, not as part of the monthly calendar. That is an **on-demand post**.

The journey is simple to describe:

- 1 The client describes what they want**
on the on-demand page and clicks *Generate*.
- 2 The request is saved**
to a table called `on_demand_requests` and handed to the automation engine (**n8n**).
- 3 A chain of AI “agents” builds the post**
— a Saudi-dialect caption, an image (or video), quality checks, and a safety decision.
- 4 One of three things happens:**
the post is **delivered** straight to the client, **held** for a human to review, or it **failed**.
- 5 If it was held,**
an OGz admin reviews it on the QA screen and either approves (releases it) or rejects it.
- 6 An email is sent**
to the brand owner at the moment their post genuinely becomes available.

The one rule that shapes everything

The company has a hard rule: **the CEO agent always decides first**. Every single generation — calendar or on-demand — is routed through the CEO agent before anything else runs. This is why you will see “CEO” as the very first step of the pipeline.

2 — The client page: `/{{brand}}/on-demand/new`

This is the screen at `http://localhost:3000/shawarmahouses-a-7psz/on-demand/new` (the first part of the URL is the brand’s unique slug). It has two parts side by side.

2.1 — The “Brand Context” card (left, read-only)

Before the form, the page shows a small card listing facts it already knows about the brand, pulled automatically from the brand’s profile (its “BrandDNA”):

- **Sector** (e.g. food & beverage)
- **Dialect** — which Arabic dialect captions should use
- **City, Channel** (e.g. Instagram), and the brand **Voice**

The client cannot edit these — they are shown so the client understands the “defaults” the AI will use. A small `< 5 min` badge sets the expectation that the post arrives quickly.

2.2 — The form (right)

Deliberately, the form asks the client for almost nothing — only two real choices:

1. **Output type** — a simple *Image* or *Video* toggle.
2. **The prompt** — a single text box where the client describes the post they want, in plain English. It must be between **10 and 5,000 characters**.

Hard Rule #3 — Arabic is never typed into the prompt

The prompt box is locked to left-to-right (English) input. Arabic text is **never** sent to the image generator. Instead, the Arabic caption is laid *on top* of the finished image afterwards. This guide notes this rule again later because it appears in several places.

2.3 — What the client doesn’t fill in (but the system does)

Everything else is filled in automatically so the client isn’t overwhelmed. When the form is submitted it quietly attaches sensible values:

Content type	<code>educational</code> (a safe default that doesn’t force a particular style)
Objective	<code>awareness</code>
Platform	taken from the brand’s primary channel (e.g. Instagram)
Canvas / shape	derived from the platform (square for Instagram, “snap” for Snapchat)
Colour palette	the brand’s primary colour
Posting time / month	“now” / the current month

The same prompt text is used for both the **style descriptor** and the **hero concept** the AI works from.

2.4 — What happens the instant the client clicks “Generate”

- 1 The form sends the request**
to the address `POST /api/posts/on-demand`.
- 2 The system checks the client owns this brand**
(security), then validates every field.
- 3 A row is created in `on_demand_requests`**
with status `queued`. This gives the request a permanent ID before anything else runs.
- 4 The system signs the request**
with a secret key and fires it to the n8n automation (the “N8N-A02” workflow), flipping the status to `generating`.
- 5 It immediately replies “accepted”**
(it does *not* wait for the post to finish). The page now starts polling for progress.

If the automation engine can't be reached, the request is marked `failed`, an anomaly record is logged for the team, and the client sees a friendly error.

2.5 — The live progress bar

While the post is being built, the page checks the status every **2.5 seconds** and shows a four-stage tracker:

Validating → Connecting → Generating → Finalizing.

Behind “Generating”, the page shows exactly which agent is working right now, because the automation reports its real step: *analysing brand context (CEO) → building caption context (COO) → writing the Arabic caption → quality checks → creating the image → saving*. If it takes unusually long (over 2 minutes for an image, 5 minutes for a video), a gentle “this is still working — check back later” message appears, but the work keeps running.

Reference: page `apps/web/src/app/[slug]/on-demand/new/page.tsx` · form `on-demand-form.tsx` · entry point `apps/web/src/app/api/posts/on-demand/route.ts` · progress polling `apps/web/src/app/api/posts/on-demand/[requestId]/status/route.ts`

3 — What happens after “Generate” (the AI pipeline)

Once n8n picks up the request, it runs a fixed sequence of specialist “agents”. Each agent is a small AI service that does one job. Think of it as an assembly line:

1

CEO — classify & route

The CEO agent looks at the brand and the request and decides how to handle it (Hard Rule #1: the CEO always goes first).

2

COO — compile caption context

Gathers the brand’s tone, dialect and rules so the caption sounds right.

3

DeepSeek — generate

Writes the Arabic caption and the English “visual brief” (the description the image model will use).

4

CEO — match chain

Reads what the post is actually about and picks the best visual “chain” (style template + image model). Covered in Section 4.

5

CCO — quality check (QC)

Scores the result: brand relevance, prompt match, brand consistency, overall quality.

6

CEO — confidence gate

The safety decision. Outputs one of three verdicts: `clean`, `watermark`, or `hold`.

7

Image / video generation (FAL AI)

The picture or clip is created. The Arabic caption is then laid on top using image software — never baked into the AI prompt (Hard Rule #3).

8

Upload to storage

The finished file is downloaded and re-uploaded to the company’s own Supabase storage (Hard Rule #4 — the temporary generator URL is never saved).

9

Callback

n8n calls back to `/api/webhooks/n8n` with the message `on_demand_complete` so the website knows it's done.

Two hard rules you'll see in the artwork

Arabic overlay (Rule #3): the admin screen actually shows a “clean version (no Arabic overlay)” alongside the final image to prove the rule was followed.

Own storage (Rule #4): only the company's storage links are ever written to the database.

Reference: the agent services live under `apps/web/src/app/api/agents/*` — e.g. `ceo/classify`, `coo/compile-caption-context`, `deepseek/generate`, `ceo/match-chain`, `cco/qc`, `ceo/confidence-gate`. The orchestration sequence is described at the top of `api/posts/on-demand/route.ts`.

4 — How chain selection works

A “chain” is a **reusable visual recipe**: a style template plus the specific image/video model and settings used to produce it. Example chains: a “Hot Food Hero” chain for steaming dishes, a “Booking CTA” chain for call-to-action posts.

4.1 — Why the chain is picked *after* the caption is written

The system deliberately does **not** guess the chain up front. It waits until DeepSeek has written what the post is actually about, then chooses the chain that best fits that real content. Picking a chain before seeing the content would be “blind”.

4.2 — The two related steps

Step	What it decides	When
Select-chains (monthly calendars)	Only the <i>format</i> (image vs video) and the <i>content type</i> for each slot — not the chain itself.	Before generation
Match-chain (the real chain picker)	Reads each finished post's actual caption + content and selects the single best-matching chain from the database.	After generation, before quality scoring

For an on-demand post, the client just types a free-text prompt and the system picks the chain (and the image model) automatically from that prompt — the client never chooses a chain.

4.3 — The clever part: the rules live in the data, not the code

Self-extending design

The chain picker has **no hard-coded list of chains**. Instead, every chain is a row in the database with its own description — its *name*, *purpose*, *intent*, *style affinity*, and *anti-patterns* (what it should NOT be used for). The AI reads those descriptions and reasons about the best match. This means an admin can **add a brand-new chain simply by adding a database row** — it becomes selectable instantly, with no code changes.

If the AI picker ever fails, the system falls back to a deterministic scoring method so a post is never left without a chain. The chosen chain’s ID and its prompt template are stored with the post and shown on the admin screen.

Reference: `apps/web/src/app/api/agents/ceo/match-chain/route.ts` (the post-generation picker) and `ceo/select-chains/route.ts` (format/content-type planner). The note “selection logic lives in the DATA, not here” is in the match-chain system prompt.

5 — The three outcomes: Delivered, Held, Failed

The CEO confidence gate (Section 3) decides which of these happens. The outcome is recorded in the `status` field of the request.

`delivered` — straight to the client

The post passed all checks. It is written into `calendar_posts` and becomes visible to the client on their on-demand list and detail page. **An email is sent immediately.** A `clean` verdict means no watermark; a `watermark` verdict means the image is delivered with a watermark applied.

`held` — sent for human review

The CEO gate (or a brand-safety trigger) decided a human must look first. A row is created in the `qa_review_queue` and the request status becomes `held`. The post does **not** appear to the client yet, and — importantly — **no “your post is ready” email is sent at this stage.** That email only fires later, if and when an admin approves it.

`failed` — something went wrong

The automation couldn't finish (a timeout, an unreachable service, a quality failure, or a hard safety block). The status becomes `failed` and a plain-language reason is shown to the client, who can edit the prompt and try again. The raw technical reason is kept for support but never shown directly to the client.

THE SAFETY NET

When n8n calls back with `on_demand_complete`, the website performs a careful “safety-net” update: it only flips a request to `delivered` if it was still `generating`. This guarantees it can **never accidentally overwrite a “held” post** and email the client about something a human hasn't approved yet.

Reference: outcome handling in `apps/web/src/app/api/webhooks/n8n/route.ts` (search for `on_demand_complete`); the status values are defined by the `on_demand_status_type` enum in `supabase/migrations/0044_on_demand_requests.sql`.

6 — The database: every table & field, and why

Five tables do the heavy lifting. Here is each one in plain terms, with what every field is for.

6.1 — `on_demand_requests` — the client's brief & the request's life story

Why it exists: the normal product is a monthly calendar of ~20 posts. An on-demand post is a single, one-off request with a custom brief that doesn't fit the monthly structure — so it gets its own table. This table captures the brief, tracks the request end-to-end, and links to the post that eventually gets produced.

Field	Plain-English meaning
<code>request_id</code>	The unique ID for this request (the page URL uses it).
<code>brand_id</code>	Which brand this belongs to.
<code>content_type</code> , <code>objective</code>	What kind of post and its goal (e.g. educational / awareness).
<code>media_type</code>	Image or video (added later, defaults to image for old rows).
<code>platform</code> , <code>canvas</code>	Where it's for (Instagram, Snapchat...) and its shape/aspect ratio.
<code>posting_time</code> , <code>month</code> , <code>posts_per_week</code>	Scheduling hints.
<code>occasion_name</code> , <code>occasion_lead_weeks</code> , <code>occasion_priority</code>	Optional event the post ties to (e.g. National Day).
<code>hashtags</code>	Requested hashtags.
<code>style_descriptor</code> , <code>hero_concept</code>	The visual brief — comes from the client's prompt (English only, Rule #3).
<code>negative_prompt</code>	What the image should avoid.
<code>cultural_guidance</code>	Cultural notes for the post.
<code>color_palette</code>	Brand colours to use (hex codes).
<code>image_model_pref</code>	Preferred image engine, or <code>auto</code> to let the system choose.
<code>first_ever_post</code>	Flags a brand's very first output (auto-held for an extra quality check).
<code>overlay_brand_name_ar</code>	Whether to overlay the Arabic brand name on the image.
<code>status</code>	The life story: <code>queued</code> → <code>generating</code> → <code>delivered</code> / <code>held</code> / <code>failed</code> .

<code>current_step</code>	The live pipeline step (ceo, coo, caption, qc, image, upload, done) — powers the progress bar.
<code>confidence_mode</code>	The confidence mode the CEO chose for this run.
<code>post_id</code>	Once finished, points to the <code>calendar_posts</code> row that was created.
<code>n8n_request_id</code>	Correlation ID so n8n's callbacks match the right request.
<code>failure_reason</code>	Why it failed, if it did.
<code>submitted_by</code>	Which user submitted it.
<code>created_at</code> , <code>updated_at</code> , <code>delivered_at</code>	Timestamps.

Security: a client can only read and insert their own brand's requests; clients can never update the status directly — only the automation (using a service-role key) can. This is enforced by database rules (RLS).

6.2 — `calendar_posts` — the actual finished post

This is the shared table for every post, whether it came from a monthly calendar or from an on-demand request. On-demand changes added a few things:

Field	Plain-English meaning
<code>calendar_id</code>	Made optional — on-demand posts have no monthly calendar.
<code>on_demand_request_id</code>	Links the post back to its on-demand brief.
<code>route_decision</code>	The CEO gate verdict made explicit: <code>clean</code> , <code>watermark</code> , or <code>hold</code> .
<code>revision_count</code>	How many times the client has regenerated this post (capped at 3).
<code>storage_url</code> , <code>clean_storage_url</code>	The final image/video, and the “no Arabic overlay” clean version.
<code>caption_ar</code> , <code>hashtags</code>	The Arabic caption and tags.
<code>confidence_score</code> , <code>watermark</code>	Quality score and whether a watermark was applied.
<code>status</code>	Where the post is in its own approval life (e.g. <code>pending</code> when released to the client).

Built-in safeguard: a rule guarantees every post belongs to *exactly one* parent — either a monthly calendar or an on-demand request, never both and never neither.

6.3 — `admin_regenerations` — the admin's private “draft lane”

Why it exists: when an *admin* regenerates a post, the new version must **not** be shown to the client until the admin approves it. So admin regenerations live in their own table as “drafts” and never touch the live post until promoted.

Field	Plain-English meaning
<code>regen_id</code>	Unique ID of this draft version.
<code>post_id</code> , <code>request_id</code> , <code>brand_id</code>	Which post / request / brand this draft is for.
<code>version</code>	Version number (v1, v2, ...) for this post.
<code>media_type</code> , <code>storage_url</code> , <code>clean_storage_url</code>	The draft's media files.
<code>caption_ar</code> , <code>hashtags</code> , <code>confidence_score</code> , <code>watermark</code>	The draft's caption, tags and scores.
<code>prompt_override</code>	If the admin edited the prompt, the custom prompt used.
<code>image_model</code>	Which model produced this draft.
<code>status</code>	<code>draft</code> → <code>approved</code> / <code>rejected</code> / <code>superseded</code> (replaced by another approved version).
<code>created_by</code> , <code>reviewed_by</code> , <code>created_at</code> , <code>reviewed_at</code>	Who made/decided it and when.

Security: this table is **admin-only** at the database level — clients have no access whatsoever, so admin drafts can never leak.

6.4 — `qa_review_queue` — the “needs a human” list

When a post is `held` , a row is added here. This is what populates the admin QA screen.

Field	Plain-English meaning
<code>queue_id</code>	Unique ID of the review item.
<code>post_id</code> , <code>brand_id</code>	Which post and brand.
<code>status</code>	<code>pending</code> → <code>approved</code> / <code>rejected</code> .
<code>trigger_reason</code>	Why it was held (the hold reasons — see Section 7.4).
<code>cco_score</code> , <code>flags</code>	The quality score and detailed flags (brand relevance, prompt match, etc.).
<code>resolved_at</code> , <code>reviewed_at</code>	When it was decided.

6.5 — Supporting tables

- `usage_logs` — an audit trail; every webhook callback and every admin approve/reject is recorded here.
- `anomaly_records` — logged when something abnormal happens (e.g. the automation engine is unreachable).
- `brand_profiles` — the brand's identity and the owner's account, used to find the email address.

- **calendars** — monthly calendars (used to look up the month for emails when relevant).

Reference: `supabase/migrations/0044_on_demand_requests.sql` (requests table + calendar_posts changes), `0047_revision_count.sql` (revision_count), `0074_on_demand_current_step.sql` (current_step), `0075_on_demand_media_type.sql` (media_type), `0099_admin_regenerations.sql` (admin draft lane). Types in `packages/db/src/types.ts` and queries in `packages/db/src/queries/on-demand.ts` & `admin-regenerations.ts`.

7 — The admin review screen: `/admin/qa?tab=on_demand`

This is the OGz team’s quality-assurance (QA) desk at `http://localhost:3000/admin/qa?tab=on_demand`.

7.1 — The two tabs

The QA page has two tabs, each showing a count of items waiting:

- **Calendar** — held posts from monthly calendars.
- **On-Demand** — held posts from on-demand requests (the focus of this guide).

The On-Demand tab shows a **paginated grid of brand cards** (10 per page) — one card per brand that has on-demand activity. A small “new” badge signals fresh arrivals. Clicking a brand opens that brand’s on-demand items, and clicking an item opens the full **inspection page**.

7.2 — The inspection page layout

The inspection page is a two-column workspace built to give a reviewer everything in one place.

LEFT COLUMN — FOUR TABS

Tab	What it shows
Preview	The generated image or video, the “clean version” (no Arabic overlay, proving Rule #3), the Arabic caption (shown right-to-left), the hashtags, and a strip of regenerated versions.
DeepSeek Data	Everything the AI produced: the client’s original prompt, the English image prompt (with a green “Arabic-clean” badge or a red warning if Arabic slipped in), the negative prompt, the chain template and chain ID, cultural guidance, a metadata grid (content type, format, platform, route decision, watermark, model, confidence mode, occasion), the colour palette, and the request ID.
CCO / Flags	The quality scores and a detailed breakdown of every hold reason, plus the raw flag values.
Revision History	Every admin regeneration draft for this post, each with its version, time, image, caption, scores and status.

RIGHT COLUMN — FACTS & ACTIONS

- **IDs card:** Post ID, Queue ID, Request ID, and created date.
- **Score cards:** CCO Score, Visual QC, Brand Relevance, Prompt Match, Brand Consistency. A dash (“—”) means that check didn’t run — scores are never invented.
- **Quick meta:** Route, Chain, Format, Platform, Watermark, Model — colour-coded (green = clean, amber = watermark, red = hold).
- **Hold reasons summary:** badges showing why it was held.
- **Action suite:** the Regenerate button and the Approve / Reject buttons.

7.3 — Selecting a regenerated version

When a reviewer clicks a draft version in the history strip, the *entire* page re-renders to show that version — its image, caption, prompts and model — so they can compare versions exactly as they'd see the original. (The original AI quality scores don't apply to a draft, so those simply show “—” for a selected draft.)

7.4 — The hold reasons (why a post is sent for review)

Each hold reason has a clear label and explanation on screen. The full set:

Reason	What it means
First-ever output	The brand's very first post — held for an initial quality audit.
CCO brave-route flag	The culture agent flagged the content as culturally risky.
Health claim	A healthcare/testimonial claim needing compliance review.
Finance claim	A financial benefit claim that must be verified.
Government sector	All government-sector content is held by policy.
High religious sensitivity	A religious reference for a high-sensitivity brand.
Dialect unconfirmed	The Arabic dialect couldn't be confirmed automatically.
Conflict record	The brand has an unresolved conflict on file.
Revision limit reached	The maximum number of revision cycles was hit.
CCO low score (<50)	Quality scored below the threshold.
Hard block	Matched a prohibited pattern — cannot be approved at all .
Method drift	The post deviated from the approved brand method.
CEO hold	The CEO routing layer flagged it on brand-risk grounds.
Admin hold	An admin manually placed it on hold.

Hard Block is special

If a post is a **Hard Block**, the page shows a red banner and the *Approve button is disabled* — there is no way for an admin to release a hard-blocked post.

Reference: tab page `apps/web/src/app/admin/qa/page.tsx` · brand list `on-demand-qa-list.tsx` · inspection screen `admin/qa/on-demand/[brandId]/on-demand-inspection-page.tsx` · the hold-reason labels are the `TRIGGER_LABELS` / `TRIGGER_DETAILS` maps in that file. Data via `adminQ.getQaOnDemandGroups()` .

8 — Regeneration (client side & admin side)

Both the client and the admin can ask the system to “try again”. They use the **same automation workflow (N8N-B03)**, but with one crucial difference — a `target` flag — that completely changes the outcome.

8.1 — Client regeneration (target = user)

On their on-demand detail page, a client can click **Regenerate** (optionally with a custom prompt). Then:

- The system first checks the post hasn’t already been regenerated **3 times** (the cap). If it has, it stops and says so.
- N8N-B03 runs the full pipeline and **overwrites the live post in place**, bumping `revision_count` by one.
- Because the live post changed, the new image is immediately visible to the client, and a “your image is ready” email is sent.

A regeneration can itself come back **held** (sent to admin review) or **rejected** (blocked by the brand-safety gate) — the client sees a clear message for each case.

8.2 — Admin regeneration (target = admin)

On the inspection page, an admin clicks **Regenerate** and chooses one of two options in a modal:

- **Use Existing Prompt** — re-run with the same prompt.
- **Edit & Regenerate** — refine the prompt first (English only).

The critical difference: the admin’s result does **not** overwrite the live post. Instead it is saved as a new **draft** in `admin_regenerations` — invisible to the client — and appears in the version-history strip. The admin can generate several drafts and compare them. A live progress tracker (Queuing → Connecting → Running AI generation → Saving draft) mirrors the real pipeline step by polling the admin status endpoint.

Why two completely separate paths?

The client’s own regeneration history (live post + `revision_count`) is kept entirely independent from the admin’s draft history. This way an admin can experiment freely without ever disturbing what the client currently sees.

Reference: client path `apps/web/src/app/[slug]/on-demand/actions.ts → requestRegenerate` · admin path `apps/web/src/app/actions/admin-regenerate.ts → requestAdminRegenerate` · admin button UI `admin/qa/admin-regenerate-button.tsx` · admin progress endpoint `api/admin/on-demand/[requestId]/status/route.ts` · workflow `n8n/flows/N8N-B03_Revision Request Handler.json`.

9 — Approve, reject & hold — what each button really does

The inspection page has **two different kinds** of approve/reject. It's important not to confuse them.

9.1 — Approving / rejecting the held item itself

These are the big Approve / Reject buttons in the bottom-right action suite. They decide the fate of the original held post:

- **Approve** releases the held post to the client and resolves the queue item. (Disabled for a Hard Block.)
- **Reject** declines it. Both actions ask for confirmation and are recorded in the audit log.

9.2 — Approving / rejecting an admin draft version

Inside the version-history strip, each admin draft has its own “**Approve this version**” and “**Reject**” controls. Approving a draft is the moment a private draft becomes the live post. Here is exactly what “approve a draft” does, step by step:

- 1 Promote the draft into the live post.**
The draft's image, caption, hashtags, score and watermark are copied into `calendar_posts`, and the post's status is set to `pending`.
- 2 Reveal the request.**
The `on_demand_requests` row is marked `delivered` so the client can see it.
- 3 Tidy the drafts.**
This draft is marked `approved`; any other open drafts for the same post are marked `superseded`.
- 4 Close the queue item.**
The matching `qa_review_queue` row is marked `approved` and leaves the queue.
- 5 Record & notify.**
The decision is written to the audit log, and the “post approved” email is sent to the client.

Important nuance: “admin approval = release, not final approval”

When an admin approves, the post is released to the client with status `pending` — meaning the **client** then approves it (or requests a revision) from their own calendar. Admin approval makes it visible; the client still has the final say on their own content.

Rejecting a draft simply marks that draft `rejected`. It stays in the history for the record, and nothing the client sees changes — they keep seeing the last approved asset.

Reference: held-item actions `apps/web/src/app/actions/qa.ts` → `approveQaItem` / `rejectQaItem` .
draft actions `apps/web/src/app/actions/admin-regenerate.ts` → `approveAdminRegeneration` / `rejectAdminRegeneration` .

10 — How email notifications work

Emails are sent through a shared helper called `notify()`, which uses **Resend** as the email provider. The system is careful to email the client only at the *genuine* moment a post becomes available — never before.

10.1 — When an email is (and isn't) sent

Event	Email sent?	Which template
Post delivered straight away	Yes — immediately	<code>post_approved</code>
Post held for review	No — held posts wait for an admin	—
Admin approves a held post / draft	Yes — at the moment of approval	<code>post_approved</code>
Client regeneration succeeds	Yes — “new image is ready”	<code>post_approved</code>
Post failed	No email — shown on screen instead	—

Notice the same `post_approved` (“your post is ready”) template is reused across all the “it’s available now” moments, so the client gets one consistent message.

10.2 — How the right person is found

To send the email, the system looks up the brand to find the owner’s account, then resolves that account’s email address. The email is personalised with the **brand name**, the post’s **position**, and the **month** (taken from the post’s calendar, or from the request’s posting time for on-demand posts).

A subtle but important technical choice

These emails are sent “awaited” (the code waits for the send to complete) rather than fire-and-forget. On serverless infrastructure, a fire-and-forget send can be killed before it finishes, leaving the notification stuck. The `notify()` helper also never throws an error, so a hiccup in email can never break the post itself.

10.3 — Internal alerts (not to clients)

Separately, when an anomaly is routed for human attention, the relevant internal “copilot” (tech, management, or production) receives a direct alert email so the team can act quickly. These go to staff, not clients.

Reference: delivery email `api/webhooks/n8n/route.ts` → `fireNotifyOnDemandDelivered` · client-regeneration email `[slug]/on-demand/actions.ts` → `fireNotifyOnDemandRegenerated` · admin-approval email `actions/admin-regenerate.ts` → `fireNotifyApproved` · the `notify()` helper and templates live in `@repo/email`.

11 — Reference map (where everything lives)

For anyone who wants to trace a behaviour back to the code, here is the full map.

CLIENT EXPERIENCE

What	File
On-demand “new” page	<code>apps/web/src/app/[slug]/on-demand/new/page.tsx</code>
The form & progress UI	<code>apps/web/src/app/[slug]/on-demand/new/on-demand-form.tsx</code>
Submit endpoint	<code>apps/web/src/app/api/posts/on-demand/route.ts</code>
Progress polling	<code>apps/web/src/app/api/posts/on-demand/[requestId]/status/route.ts</code>
Client regenerate action	<code>apps/web/src/app/[slug]/on-demand/actions.ts</code>

ADMIN EXPERIENCE

What	File
QA tabs page	<code>apps/web/src/app/admin/qa/page.tsx</code>
On-demand brand list	<code>apps/web/src/app/admin/qa/on-demand-qa-list.tsx</code>
Inspection page	<code>apps/web/src/app/admin/qa/on-demand/[brandId]/on-demand-inspection-page.tsx</code>
Admin regenerate button	<code>apps/web/src/app/admin/qa/admin-regenerate-button.tsx</code>
Version-history strip	<code>apps/web/src/app/admin/qa/admin-regen-history.tsx</code>
Admin regenerate / approve / reject	<code>apps/web/src/app/actions/admin-regenerate.ts</code>
Held-item approve / reject	<code>apps/web/src/app/actions/qa.ts</code>
Admin progress polling	<code>apps/web/src/app/api/admin/on-demand/[requestId]/status/route.ts</code>

AUTOMATION, DATA & RULES

What	File
n8n callback receiver	<code>apps/web/src/app/api/webhooks/n8n/route.ts</code>
Regeneration workflow	<code>n8n/flows/N8N-B03_Revision_Request_Handler.json</code>
Chain pickers	<code>api/agents/ceo/match-chain</code> , <code>ceo/select-chains</code>
AI agents	<code>apps/web/src/app/api/agents/*</code> (ceo, coo, deepseek, cco)

Requests table & calendar_posts changes	<code>supabase/migrations/0044_on_demand_requests.sql</code>
Revision count	<code>supabase/migrations/0047_revision_count.sql</code>
Live step / media type	<code>0074_on_demand_current_step.sql</code> , <code>0075_on_demand_media_type.sql</code>
Admin draft lane	<code>supabase/migrations/0099_admin_regenerations.sql</code>
Database types & queries	<code>packages/db/src/types.ts</code> , <code>queries/on-demand.ts</code> , <code>queries/admin-regenerations.ts</code>
Company hard rules	<code>CLAUDE.md</code> (Hard Rules #1–#4)

This document was produced by reading the live source code directly. Status values, table fields, hold reasons and email behaviour reflect the code as it stands in this repository on 13 June 2026. Where the code contains a safeguard (such as the “never overwrite a held post” guard, or the disabled Approve button on a Hard Block), it has been described exactly as implemented rather than assumed.